Exploratory Data Analysis (EDA) on Sales Dataset Using Python

1. Introduction

In today's data-driven world, businesses collect large volumes of sales data. Analysing this data helps organizations understand customer behavior, product performance, and revenue trends. Exploratory Data Analysis (EDA) is a crucial step in data analysis that helps summarize and visualize data to uncover patterns and insights.

In this project, a sales dataset is analysed using Python and its data analysis libraries. The analysis focuses on identifying revenue trends, understanding product performance, and visualizing sales patterns using different charts.

2. Problem Statement

Businesses generate large amounts of sales data every day. However, raw data alone does not provide meaningful insights. Without proper analysis, it becomes difficult to identify important patterns such as sales trends, high-performing products, and overall revenue distribution.

The problem addressed in this project is to analyse a sales dataset and extract useful insights using data cleaning, statistical analysis, and visualization techniques. By performing Exploratory Data Analysis (EDA), the project aims to transform raw sales data into meaningful information that can help understand business performance and support better decision-making.

3. Objectives

The main objectives of this project are:

- To load and analyse a sales dataset using Python.
- To clean the dataset by handling missing values.
- To calculate important sales metrics such as total and average revenue.
- To visualize the sales data using different graphs.
- To identify patterns and trends in product sales.
- To generate meaningful insights that help understand business performance.

4. Tools and Technologies Used

Programming Language

- Python

Libraries Used

- **Pandas** – Data manipulation and analysis
- **NumPy** – Numerical operations
- **Matplotlib** – Data visualization
- **Seaborn** – Advanced statistical visualizations

Software

- **Visual Studio Code –** used to write and run the project
- **Jupyter Notebook** – used for performing EDA and visualization
- **Python Environment** (Anaconda / Python)

5. System Flow

The following steps describe the workflow used in this project:

1. Import required Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn.
2. Load the dataset using the Pandas read_csv() function.
3. Display the dataset to understand its structure.
4. Check for missing values in the dataset.
5. Perform data cleaning by removing or handling missing values.
6. Convert the Date column into datetime format for time analysis.
7. Calculate important metrics such as total revenue and average revenue.
8. Group data by product to determine product-wise revenue.
9. Create visualizations including:
    - Line chart for sales trends over time
    - Bar chart for product-wise revenue
    - Pie chart for revenue distribution
10. Analyze the visualizations and write insights based on the observed patterns.

6. Implementation Steps

Step 1: Import Libraries

The required Python libraries are imported to perform data analysis and visualization.

Step 2: Load the Dataset

The dataset is loaded into the Jupyter Notebook using the Pandas read_csv() function.

Step 3: Explore the Dataset

The dataset structure is examined using functions such as:

- head()

These functions help understand the columns.

Step 4: Data Cleaning

Missing values in the dataset are handled by removing incomplete rows. The Date column is converted into datetime format for proper time-based analysis.

Step 5: Calculate Key Metrics

Important metrics such as total revenue and average revenue are calculated using the dataset. Product-wise revenue is also calculated using group-by operations.

Step 6: Data Visualization

Several graphs are created to visualize the sales data:

- **Line Chart** – Displays sales trends over time.
- **Bar Chart** – Shows revenue generated by each product.
- **Pie Chart** – Displays revenue share distribution among products.

These visualizations make it easier to understand patterns and trends in the data.

7. Output Screenshots

The following screenshots are included in the project:

1. Libraries and Dataset preview (first 8 rows of the dataset)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

sns.set_style('whitegrid')  # Makes plots look better
```
✓ 0.0s

```
df = pd.read_csv("sales_dataset_1000_rows.csv")
df.head(8)
```
✓ 0.0s

	Date	Product	Category	Quantity	Unit_Price	Discount	Revenue	Region	Sales_Channel	Customer_Type
0	2024-09-27	Keyboard	Accessories	4.0	50.0	0.00	200.0	East	Retail Store	New
1	2024-08-02	Smartwatch	Accessories	7.0	250.0	0.05	1662.5	East	Online	Returning
2	2024-05-31	Smartwatch	Accessories	4.0	250.0	0.15	850.0	East	Retail Store	New
3	2024-10-20	Smartphone	Electronics	8.0	700.0	0.15	4760.0	South	Retail Store	Returning
4	2024-11-09	Smartwatch	Accessories	NaN	NaN	0.15	212.5	South	Online	Returning
5	2024-02-28	Printer	Electronics	1.0	200.0	0.10	180.0	East	Retail Store	Returning
6	2024-09-27	External Hard Drive	Storage	12.0	100.0	0.15	1020.0	South	NaN	New
7	2024-02-24	Router	Networking	3.0	80.0	0.15	204.0	West	Retail Store	New

Generate + Code + Markdown

2. Dataset information showing column details and missing values

```
df.isnull().sum()
```
✓ 0.0s

```
Date              30
Product           30
Category          30
Quantity          30
Unit_Price        30
Discount          30
Revenue           30
Region            30
Sales_Channel     30
Customer_Type     30
dtype: int64
```

3. Cleaned dataset after handling missing values

```
df_clean = df.dropna().copy()
df_clean['Date'] = pd.to_datetime(df_clean['Date'])

# 3. Check for missing values again
print(df_clean.isnull().sum())
```

186] ✓ 0.0s

```
Date             0
Product          0
Category         0
Quantity         0
Unit_Price       0
Discount         0
Revenue          0
Region           0
Sales_Channel    0
Customer_Type    0
dtype: int64
```

4. Output showing total revenue and average revenue

```
total_sales = round(df_clean['Revenue'].sum(), 2)
average_sales = round(df_clean['Revenue'].mean(), 2)

print("Total Revenue:", total_sales)
print("Average Revenue:", average_sales)

# Revenue by product
product_sales = df_clean.groupby('Product')['Revenue'].sum().round(2)

print(product_sales)
```

✓ 0.0s

```
Total Revenue: 1347830.5
Average Revenue: 1826.33
Product
Camera                 158880.0
External Hard Drive     34320.0
Headphones              46987.5
Keyboard                18135.0
Laptop                 305865.0
Microphone              44214.5
Monitor                132300.0
Mouse                    9390.0
Printer                 62900.0
Router                  24880.0
Smartphone             246400.0
Smartwatch              79312.5
Speaker                 42000.0
Tablet                 134940.0
USB Drive                7306.0
Name: Revenue, dtype: float64
```

5. Line chart displaying sales trend over time

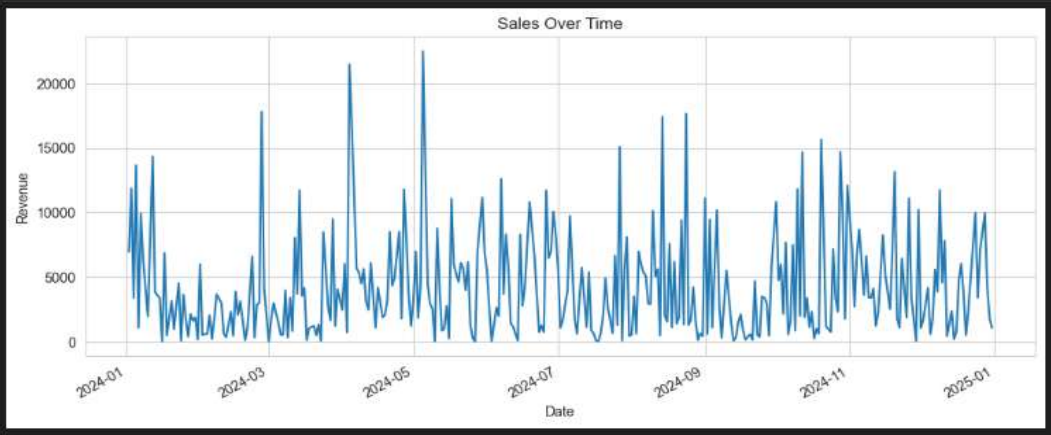
```
sales_by_date = df_clean.groupby('Date')['Revenue'].sum()

plt.figure(figsize=(12,5))
sales_by_date.plot()

plt.title("Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Revenue")

plt.show()
```

✓ 0.2s



6. Bar chart showing product-wise revenue

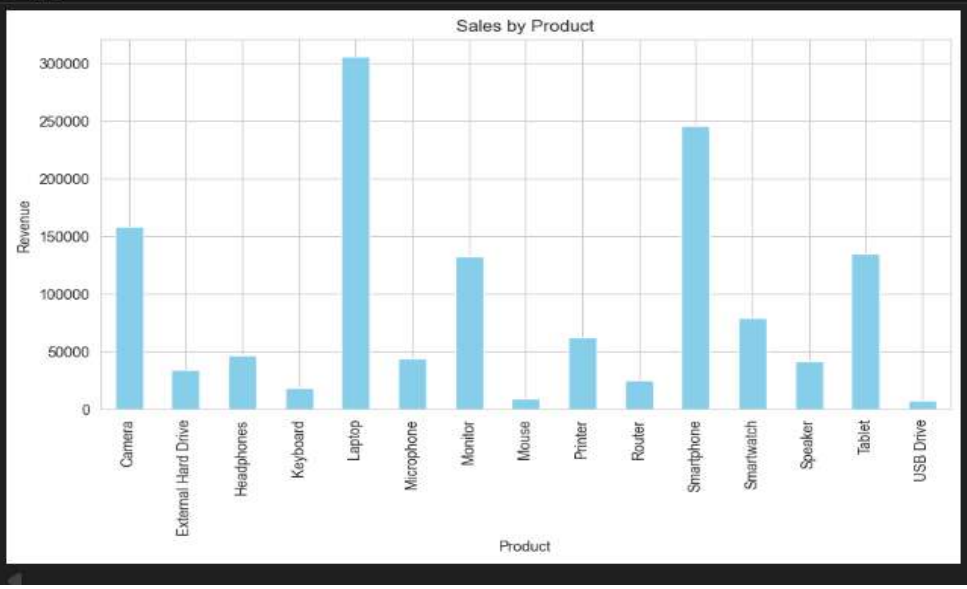
```
plt.figure(figsize=(10,5))

product_sales.plot(kind='bar', color='skyblue')

plt.title("Sales by Product")
plt.ylabel("Revenue")

plt.show()
```

✓ 0.2s



7. Pie chart showing revenue distribution

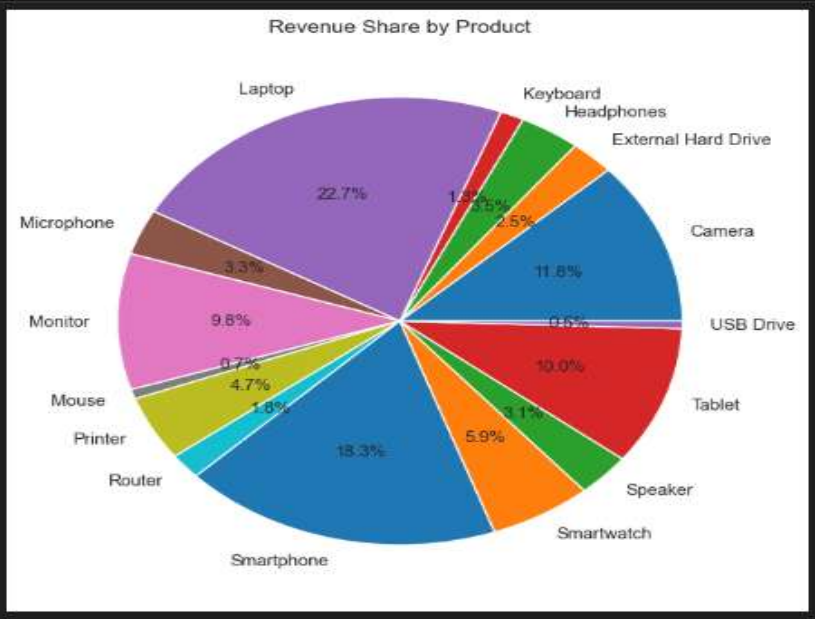
```
plt.figure(figsize=(7,7))

product_sales.plot(kind='pie', autopct='%1.1f%%')

plt.title("Revenue Share by Product")
plt.ylabel("")

plt.show()
```

✓ 0.3s



8. Insights generated from the analysis

Insights

1. The analysis shows that some products contribute significantly more revenue than others.
2. The bar chart helps identify the top-performing product in terms of sales.
3. The pie chart shows the percentage share of revenue generated by each product.
4. The sales trend over time indicates fluctuations in daily sales performance.
5. Businesses can focus on high-performing products to maximize revenue.
6. Data visualization helps understand patterns and make better business decisions.

8. Insights

1. Some products generate significantly higher revenue compared to others.
2. The bar chart clearly identifies the top-performing product.
3. The pie chart shows the percentage contribution of each product to the total revenue.
4. The sales trend line chart indicates fluctuations in revenue over time.
5. High-performing products contribute the largest share of total sales.
6. Visual analysis helps businesses identify profitable products and plan strategies accordingly.

9. Conclusion

This project successfully performed Exploratory Data Analysis on a sales dataset using Python. The dataset was cleaned and analysed to extract useful business insights. Key metrics such as total revenue and average revenue were calculated, and several visualizations were created to understand sales patterns.

The analysis showed how different products contribute to overall revenue and how sales change over time. These insights can help businesses make informed decisions regarding product focus, inventory management, and marketing strategies. Overall, this project demonstrates the complete data analysis workflow including dataset creation, data preprocessing, visualization, and insight generation using Python.